Testing

SWAT+ ships a scenario test suite that runs the model against reference input datasets and compares the output files to known-good outputs. The tests catch regressions when source changes shift numeric results outside an accepted tolerance.

Tests are wired into the CMake build and driven by `ctest`.

How it works

The test data lives in `data/`. Each subfolder is a scenario containing input files and the golden output files. The reference output files are committed copies of model output that was reviewed and accepted as correct.

The driver script `test/check.py` does the actual comparison. The wiring sits in `CMakeLists.txt`:

```
find_package(Python REQUIRED)

set(check_py "${PROJECT_SOURCE_DIR}/test/check.py")
set(exe_path "${PROJECT_BINARY_DIR}/${SWATPLUS_EXE}")
set(test_dir "${PROJECT_BINARY_DIR}/data")
set(ref_dir  "${PROJECT_SOURCE_DIR}/data")

# error tolerances
set(rel_err "0.01")
set(abs_err "1e-8")

add_test(Ithaca_sub6 python3 ${check_py} ${exe_path} ${ref_dir}/Ithaca_sub6 ${test_dir}
 ↪  ${abs_err} ${rel_err})
add_test(Ames_sub1   python3 ${check_py} ${exe_path} ${ref_dir}/Ames_sub1   ${test_dir}
 ↪  ${abs_err} ${rel_err})
```

Two scenarios are registered by default: `Ames_sub1` and `Ithaca_sub6`. Default tolerances are 1% relative error and `1e-8` absolute error. Adding another scenario means adding another `add_test` line that points at a new subfolder in `data/`.

For each scenario, `check.py`:

1. Copies the scenario folder from `data/` into the build directory.
2. Runs the SWAT+ executable inside that copy.
3. Reads `.testfiles.txt` in the scenario, which lists the output file names to check (for example, `wb.txt` and `soc.txt`).
4. For each listed file, compares the new output to the golden copy line by line and column by column. Only floating-point values on matching lines and columns are compared.
5. Two values are considered equal if:

   ```
   abs(v1 - v2) <= abs_err + rel_err * abs(v2)
   ```

6. Prints the count of differing values and the maximum relative and absolute error. The scenario fails if any pair of values does not satisfy the tolerance.

Running the tests

Python 3 has to be on `PATH`. From the `swatplus` directory:

```
cmake -B build
cmake --build build
cd build
ctest
```

`ctest` prints a summary at the end. Detailed per-test logs land in `build/Testing/Temporary/`. The most recent run is captured in `LastTest.log`.

To run a subset:

```
ctest -R Ames_sub1
```

To see test output as it runs:

```
ctest --output-on-failure
```

`make test` from inside `build/` works as an alias.

Adding a new scenario

1. Create a new subfolder under `data/`, for example `data/MyBasin`. Put a full SWAT+ input set in it.
2. Run the model in that folder and review the outputs you want to lock down.
3. Commit those output files alongside the inputs.
4. Add a `.testfiles.txt` listing the output filenames (one per line) that `check.py` should compare.
5. Add an `add_test` line to `CMakeLists.txt`.
6. Reconfigure (`cmake -B build`) and run `ctest`.

CI

The same tests run in GitHub Actions on every push to `main`. See `.github/workflows/test.yml` in the source repository.